

Exp. No: 06

IMPLEMENTATION OF MULTISTAGE GRAPH.

Date:
24/3/23

AIM:

To write a C++ program to perform multistage path graph operational forward (Approach) using dynamic programming technique and analyze the time & space complexity.

ALGORITHM:

F Graph(G, R, n, p)

// input in a k -stage graph $G = (V, E)$

// indexed in order of stages E is set of edges.

// is the cost of $[i, j]$, $P[1:k]$ is a minimum cost.

\$ cost[n] := 00;

For $j := n-1$ to 1 step -1 do

\$ // compute cost[j]

Let e to be vertex such that $\langle j, e \rangle$ is if G and $C[j:e] + \text{cost}[e]$ is minimum;

cost[j] := $C[j:e] + \text{cost}[e]$;

$d[j] := e$;

}

// find a minimum cost path
 $P[1] := 1$; $P[k] = n$;

NAME:KEERTHANAA B; REG.NO:21TD0035; YEAR/SEM/SEC:
Y2/S4/A

MULTISTAGE GRAPH

```
#include<iostream.h>
#include<conio.h> Using namespace std;
struct fwd
{
int l;
int a[20];
};
struct fwd s1[10];
void main()
{
int k,i,j,n,p[10],m,z,cost[50],v,c[20][20];
clrscr();
cout<<"enter the no of stages:";
cin>>k;
n=0;
for(i=1;i<=k;i++)
{
cout<<"enter no of vertices in stages"<<i<<":";
cin>>s1[i].l;
n+=s1[i].l;
for(j=1;j<=s1[i].l;j++)
{
cout<<"enter the value of vertex"<<j<<":";
cin>>s1[i].a[j];
}
}
for(i=1;i<k;i++) double
{
for(j=s1[i].a[1];j<=s1[i].a[s1[i].l];j++)
{
for(z=s1[i+1].a[1];z<=s1[i+1].a[s1[i+1].l];z++)
{
cout<<"\n enter the cost of c["<<j<<"]["<<z<<":";
cin>>c[j][z];
}
}
}
cost[n]=0;
```

```

int min,d[50],t;
for(i<=k-1;i>=1;i--)

{
for(j=s1[i].a[1];j<=s1[i].a[s1[i].l];j++)
{
min=999;
for(z=s1[i+1].a[1];z<=s1[i+1].a[s1[i].l];z++)
{
if(cost[z]+c[j][z]<min)
min=cost[z]+c[j][z];
t=z;
}
cost[j]=min;
d[j]=t;
}
}
p[1]=1;
p[k]=n;
for(i=2;i<k;i++)
{
p[i]=d[p[i-1]];
}
for(i=1;i<k;i++)
{
cout<<p[i]<<"---";
}
cout<<p[k];
getch(); return 0;
}

```

OUTPUT:

```
CATOBIN\TC.EXE
*** MULTISTAGE GRAPH***
*
enter the no of stages:2
enter no of vertices in stages1:2
enter the value of vertex1:14
enter the value of vertex2:15
enter no of vertices in stages2:2
enter the value of vertex1:18
enter the value of vertex2:19

enter the cost of c[14][18]:13
enter the cost of c[14][19]:17
enter the cost of c[15][18]:19
enter the cost of c[15][19]:20
1---4_
```

Result :

~~24/3/23~~ Thus the C++ program for Multistage graph has been verified and executed successfully.



AIM:

To write a C++ program to perform 0/1 knapsack problem using dynamic programming technique & analyse the time & space complexity.

ALGORITHM:

Algorithm DKnap(P, W, x, n, m)

§

// pair[] is an array of PW's

$b[0] := 1$; pair[1].p := pair[1].w = 0.
0; // s

$t := 1$; $h := 1$; // start and end of s

$b[1] := \text{next} := 2$; // next free spot in pair[]

For $i := 1$ to $n-1$ do

§ // generate s^i

$k := t$;

$u := \text{largest}(\text{pair}, w, t, h, i, m)$;

for $j = t$ to u do

§

// generate s^{i-1} and merge

$pp := \text{pair}[i].p + p[i]$; $ww := \text{pair}[i].w + w[i]$;

// (pp, ww) in the next element in s^{i-1} .

NAME:KEERTHANAA B; REG.NO:21TD0035; YEAR/SEM/SEC:
Y2/S4/A

0/1 KNAP SACK PROBLEM

```
#include<iostream.h>
#include<conio.h>
int c,z[20],t[50][50],s[20],i,j,pt[20],wt[20],x[20],m,n;
class knp
{
public:
void get();
void table();
void select();
void display();
}k;
void knp::get()
{
cout<<"\n\t0\1 KNAPSACK PROBLEM..";
cout<<"\n enter the capacity of the bag---";
cin>>m;
cout<<"\n enter the no of objects--";
cin>>n;
for(i=1;i<=n;i++)
{
cout<<"\n enter the weight of object"<<i<<" ...";
cin>>wt[i];
cout<<"\n enter the profit of object"<<i<<" .....";
cin>>pt[i];
}
}
void knp::table()
{
for(i=1;i<=n;i++)
{
for(j=0;j<=m;j++)
{
if((i==1)&&(j<wt[i]))
t[i][j]=0;
if((i==1)&&(j>=wt[i]))
t[i][j]=pt[i];
if((i>1)&&(j<wt[i]))
t[i][j]=t[i-1][j];
if((i>1)&&(j>=wt[i]))
```

```

t[i][j]=pt[i]+t[i-1][j]-wt[i];
}
}
cout<<"\n\t\t table\n";
for(i=1;i<=n;i++)
{
for(j=0;j<=m;j++)
cout<<t[i][j]<<"\t";
cout<<"\n";
}
}
void knp::select()
{
int q=0;
c=0;
int a,b,d;
i=n;
for(j=m;j>=0;j--)
{
if(t[i][j]==t[i-1][j])
{
a=i-1;
if(t[a][j]==t[a-1][j]-wt[a]+pt[a])
{
}
else
{
z[c]=t[a][j];
c++;
s[q]=a;
q++;
}
b=a-1;
d=j-wt[a];
if(t[a-1][d]==t[b-1][d-wt[b]])
{
}
else
{
z[c]=t[a-1][j-wt[a]];
c++;
s[q]=a-1;
q++;
}
}
}

```

--	--	--

```

    }
    }
    else
    {
        a=i-1;
        if(t[a][j]==t[a-1][j-wt[a]]+pt[a])
        {
        }
        else
        {
            z[c]=t[a][j];
            s[q]=a;
            q++;
        }
        b=a-1;
        d=j-wt[a];
        if(t[a-1][d]==t[b-1][d-wt[b]])
        {
        }
        else
        {
            z[c]=t[a-1][j-wt[a]];
            c++;
            s[q]=a-1;
            q++;
        }
    }
    j=d+1;
}

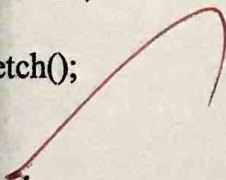
void knp::display()
{
    cout<<"\n the objects selected are---";
    for(int x=0;x<c;x++)
    {
        cout<<"\t"<<s[x];
    }
}

void main()
{
    char ch='y';
    while(ch=='y')
    {

```

--	--	--

```
clrscr();  
k.get();  
k.table();  
k.select();  
k.display();  
cout<<"\n do you want to continue(y\n)";  
cin>>ch;  
}  
getch();  
}
```



OUTPUT:

```
C:\C++\BIN\TC.EXE

00 KNAPSACK PROBLEM..
enter the capacity of the bag---20
enter the no of objects--2
enter the weight of object1...12
enter the profit of object1.....14
enter the weight of object2...15
enter the profit of object2.....17

table
0      0      0      0      0      0      0      0      0      0
0      0      14     14     14     14     14     14     14     14
14
0      0      0      0      0      0      0      0      0      0
0      0      14     14     14     17     17     17     17     17
17

the objects selected are--- 0      1      0
do you want to continue(y
```

Result:

Thus the C++ program for 0/1 Knapsack problem has been verified and executed successfully.

24/3/23

Exp. No: 08

IMPLEMENTATION OF ALL PAIRS SHORTEST PATH.

Date:
21/4/23

AIM:

To write a C++ program to perform all pairs shortest path using dynamic programming technique & analyze the time and space complexity.

ALGORITHM:

All paths (cost, A, n)

// cost[c:n; c:n] is a cost adjacently matrix of the graph with n vertices.

// A[i, j] is the cost of the shortest path for vertex i to vertex j: cost[i, j] = 0.0

for $i \leq 1 \leq n$

{ for $i := 1$ to n do

for $j := 1$ to n do

$A[i, j] = \text{cost}[i, j]$

for $k = 1$ to n do.

for $i = 1$ to n do.

for $j = 1$ to n do.

$A[i, j] = \min [a[i, j], A[i, k] + A[k, j]]$

}

NAME:KEERTHANAA B; REG.NO:21TD0035; YEAR/SEM/SEC:
Y2/S4/A

ALL PAIRS SHORTEST PATH

```
#include<iostream.h>
#include<conio.h>
int a[20][20],cost[20][20],i,j,n,k;
class path
{
public:
void get();
void allpair(int a[20][20],int n);
void put();
};
void path::get()
{
cout<<"\n\t ALL PAIR SHORTEST PATH\n";
cout<<"\n enter the no of vertices";
cin>>n;
cout<<"\n enter the cost";
for(i=1;i<=n;i++)
{
for(j=1;j<=n;j++)
{
if(i==j)
{
cost[i][j]=0;
a[i][j]=cost[i][j];
}
else
{
cout<<"\n cost["<<i<<"]["<<j<<"]="";
cin>>cost[i][j];
a[i][j]=cost[i][j];
}
}
}
cout<<"\n";
}
void path::allpair(int a[20][20],int n)
{
for(k=1;k<=n;k++)
{
```

--	--	--

```

for(i=1;i<=n;i++)
{
for(j=1;j<=n;j++)
{
if((a[i][k]+a[k][j])<a[i][j])
{
a[i][j]=a[i][k]+a[k][j];
}
}
}
}

void path::put()
{
cout<<"\n OUTPUT";
cout<<"\n all pair shortest path\n";
for(i=1;i<=n;i++)
{
for(j=1;j<=n;j++)
{
cout<<a[i][j];
cout<<"\t";
}
cout<<"\n";
}
}

void main()
{
path p;
clrscr();
p.get();
p.allpair(a,n);
p.put();
getch()
}

```

OUTPUT:

```
C:\CUBINTC\EXE
ALL PAIR SHORTEST PATH
enter the no of vertices3
enter the cost
cost[1][2]=23
cost[1][3]=27
cost[2][1]=29
cost[2][3]=13
cost[3][1]=28
cost[3][2]=39

OUTPUT
all pair shortest path
0      23      27
29     0       13
28     39     0
```

Result:

Thus the C++ program for All Pairs Shortest Path has been Verified and executed successfully.

21/4/23

